DOCUMENTATION

REAL-TIME AI ASSISTANT

The Real-Time AI Assistant is a Retrieval-Augmented Generation (RAG) based chatbot that combines a local large language model with live web search to deliver accurate and up-to-date responses. Built using Streamlit for the interactive user interface, LangChain for chaining logic, DuckDuckGo for real-time information retrieval, and Ollama to run the LLaMA 3 model locally, the system ensures responses are grounded in current search results rather than relying solely on pre-trained knowledge. It maintains conversation history using session state, allowing smooth multi-turn interactions. By integrating live search with AI-generated answers, this assistant provides reliable, context-aware, and dynamic responses, making it suitable for real-time question answering applications and AI-driven web tools.

Streamlit-Based Ollama RAG Chatbot Setup

```
import streamlit as st

from langchain_ollama import OllamaLLM
from langchain_community.tools import DuckDuckGoSearchRun
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough


#page config

st.set_page_config(page_title="Ollama RAG Chatbot",layout="centered")

st.title("Real-time AI Assistant")
st.caption("Powered by Ollama + DuckDuckGo + LangChain")

#initialize LLM& tools

llm=OllamaLLM(model="llama3:8b")
search=DuckDuckGoSearchRun()
```

This code initializes a simple Real-time AI Assistant web application using Streamlit, integrated with Ollama and LangChain components. It begins by importing necessary libraries, including `streamlit` for building the web interface, `OllamaLLM` from LangChain for running a local LLM model (`llama3:8b`), `DuckDuckGoSearchRun` for enabling web search functionality, and prompt/runnable utilities for building chains. The `st.set_page_config()` function configures the page title and layout, while `st.title()` and `st.caption()` define the app's header and description. In the initialization section, the `OllamaLLM` object loads the specified model to generate responses, and `DuckDuckGoSearchRun()` sets up a search tool for retrieving real-time information from the web. Overall, this snippet forms the foundation of a Retrieval-Augmented Generation (RAG) chatbot that combines a local language model with live search capabilities in a Streamlit interface.

Prompt Template and RAG Chain Creation in LangChain

```
prompt=ChatPromptTemplate.from_template("""
    You are a helpful AI assistant. You must answer the user's question
    based *only* on the following search results.
    If the search results are empty or do not contain the answer,
    say "I could not find any information on that."

    Search Results:
    {context}

    Question:
    {question}
    """
)

#chain creation
chain=(
    RunnablePassthrough.assign(
        context=lambda x: search.run(x["question"])
        )
        | prompt
        | llm
)
```

This code defines how your Retrieval-Augmented Generation (RAG) chatbot thinks and processes information before generating a response. First, `ChatPromptTemplate.from_template("""...""")` creates a structured prompt that instructs the AI to answer strictly based on the provided search results (`{context}`) and not from its own knowledge. It clearly tells the model to respond with "I could not find any information on that." if the search results are empty or irrelevant, which prevents hallucination. The placeholders `{context}` and `{question}` are dynamic variables that will be filled at runtime. Next, the `chain` is created using

`RunnablePassthrough.assign()`, which takes the user's question and automatically runs a DuckDuckGo search (`search.run(x["question"])`) to generate the `context`. This context is then passed into the prompt template, and finally sent to the LLM (`llm`) to generate the answer. The pipe operator (`|`) connects each component sequentially—first enriching the input with search results, then formatting the prompt, and finally generating the response—forming a complete RAG pipeline where search retrieval directly informs the model's output.

Streamlit Chat UI and Session State Handling in the RAG Chatbot

```
if "messages" not in st.session_state:
    st.session_state.messages=[]

for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])


user_query=st.chat_input("Ask me anything...")

if user_query:

    st.session_state.messages.append(
        {"role":"user","content": user_query}
    )
    with st.chat_message("user"):
        st.markdown(user_query)

    with st.chat_message("assistant"):
        with st.spinner("Thinking..."):
            response=chain.invoke({"question":user_query})
            st.markdown(response)

    st.session_state.messages.append(
        {"role":"assistant","content":response}

    )
```

This code manages the interactive chat interface and conversation memory in your Streamlit-based RAG chatbot. It first checks whether a `messages` list exists inside `st.session_state`; if not, it initializes it to store the conversation history. This ensures that messages persist across app reruns (since Streamlit reruns the script on every interaction). The `for` loop then displays all previous messages using `st.chat_message()`, rendering each message according to its role (`user` or `assistant`) and content using Markdown formatting. The `st.chat_input()` function captures new user input, and when a query is submitted, it appends the user message to session state and immediately displays it in the chat interface. Next, inside an assistant chat block, a loading spinner ("Thinking...") appears while the RAG `chain.invoke()` method processes the query by performing search retrieval and generating a response from the LLM. Once the response is generated, it is displayed in the interface and stored in `st.session_state.messages`, ensuring the assistant's reply becomes part of the ongoing conversation history. Overall, this section implements real-time conversational flow, memory persistence, and dynamic response generation in the chatbot UI.

Real-Time AI Assistant

This image shows the working output of your Real-time AI Assistant built using Streamlit, Ollama, DuckDuckGo, and LangChain. The interface displays a clean chat-style layout where user messages appear in one bubble and assistant responses appear in another. When the user asks "what is the weather today," the system retrieves live search results and generates a grounded response mentioning weather details for Thrissur, Kerala. Similarly, when the user asks "who is Indian President," the assistant responds with information retrieved through the search-based RAG pipeline, stating that Droupadi Murmu is the current President of India. The presence of contextual phrases like "Based on the search results" confirms that the model is using retrieved data rather than its internal memory. The input box at the bottom allows continuous interaction, and the conversation history is preserved using Streamlit's session state, making the chatbot function like a real conversational AI system.